

# Pipelining

---

ANISHA M. LAL

# Pipelining

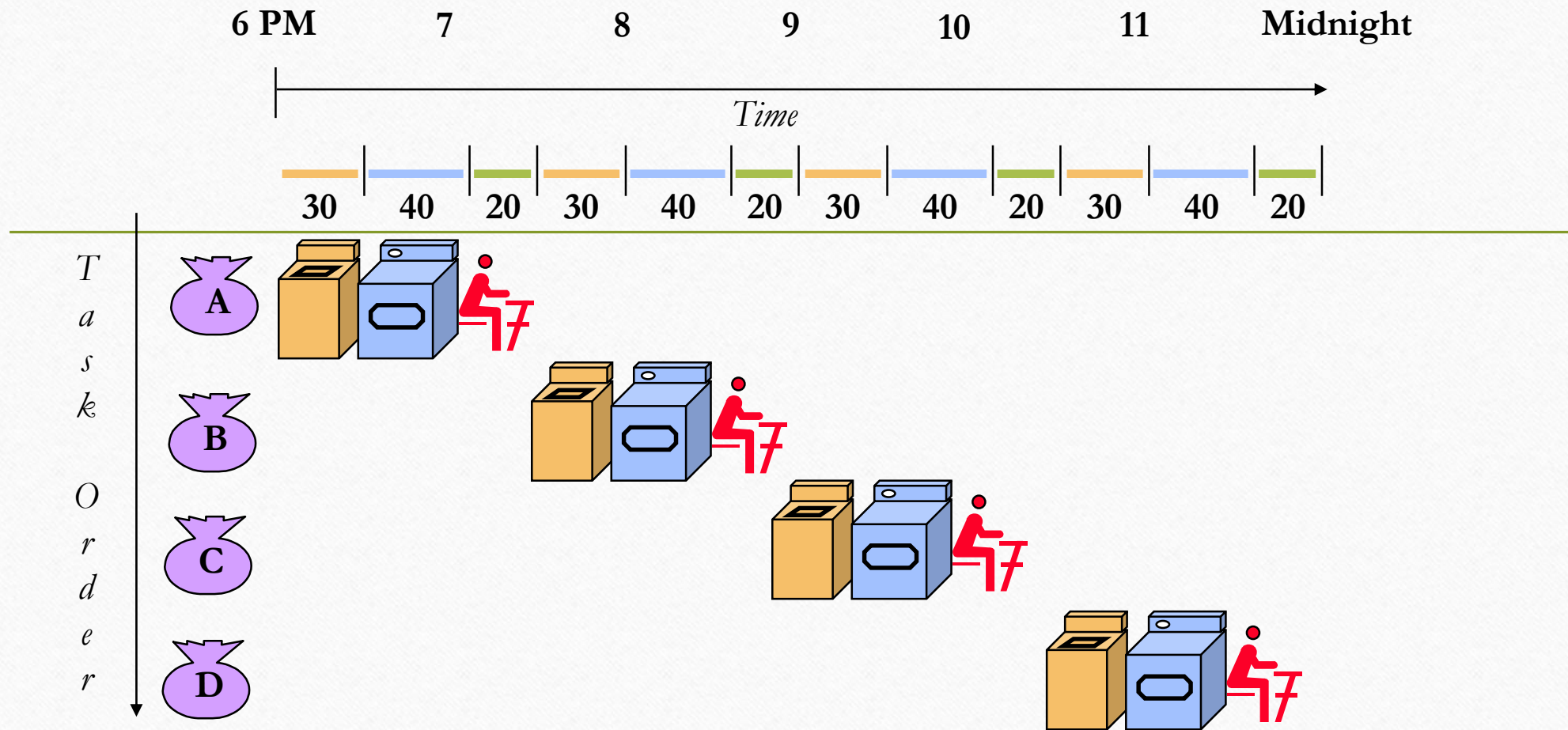
- A technique used in advanced microprocessors where the microprocessor begins executing a second instruction before the first has been completed.
- A Pipeline is a series of stages, where some work is done at each stage. The work is not finished until it has passed through all stages.
- With pipelining, the computer architecture allows the next instructions to be fetched while the processor is performing arithmetic operations, holding them in a buffer close to the processor until each instruction operation can be performed.

# How it works

---

- The pipeline is divided into segments and each segment can execute its operation concurrently with the other segments. Once a segment completes an operation, it passes the result to the next segment in the pipeline and fetches the next operation from the preceding segment

# What Is Pipelining

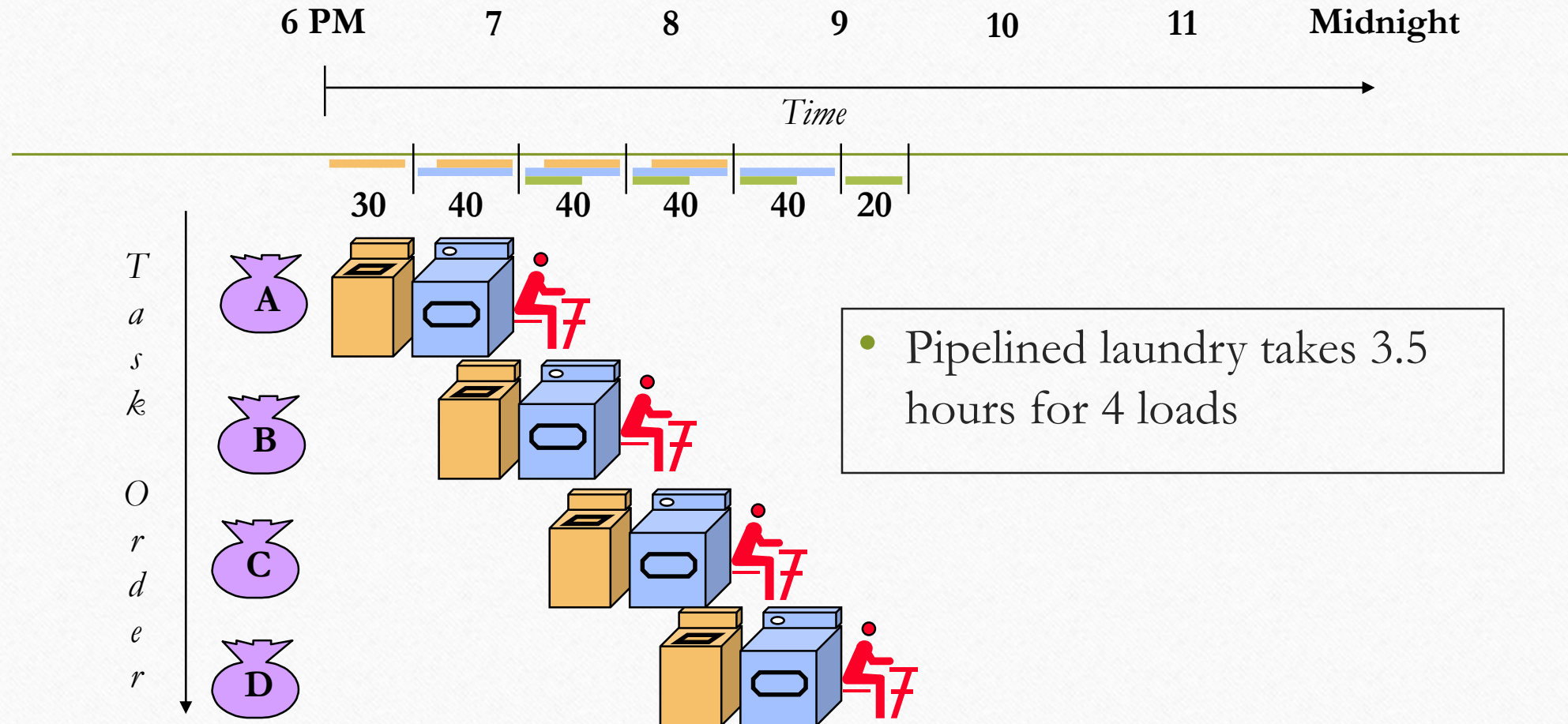


Sequential laundry takes 6 hours for 4 loads

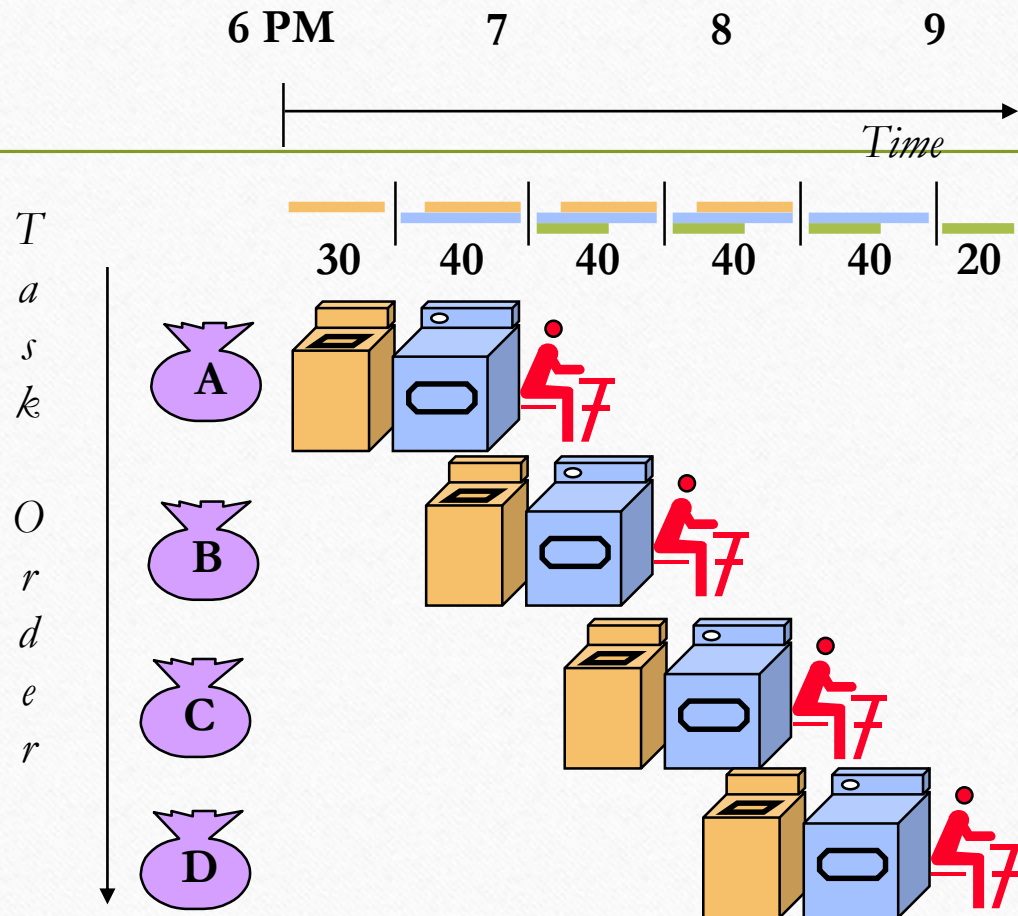
If they learned pipelining, how long would laundry take?

# What Is Pipelining

Start work ASAP



# What Is Pipelining

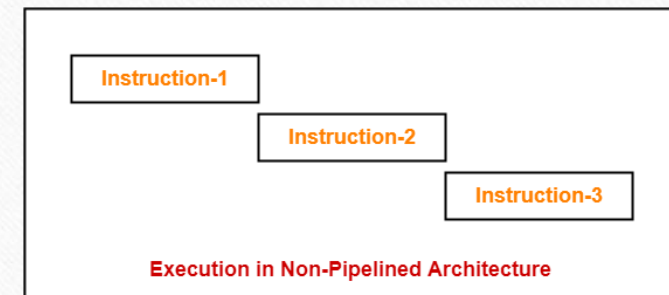


- Pipelining doesn't help latency of single task, it helps throughput of entire workload
- Pipeline rate limited by slowest pipeline stage
- Multiple tasks operating simultaneously
- Potential speedup = Number pipe stages
- Unbalanced lengths of pipe stages reduces speedup
- Time to "fill" pipeline and time to "drain" it reduces speedup

# Non-pipelined architecture

---

- In non-pipelined architecture,
- All the instructions of a program are executed sequentially one after the other.
- A new instruction executes only after the previous instruction has executed completely.
- This style of executing the instructions is highly inefficient.
- If time taken for executing one instruction =  $t$ , then,
- Time taken for executing 'n' instructions =  $n \times t$



# Pipelined architecture

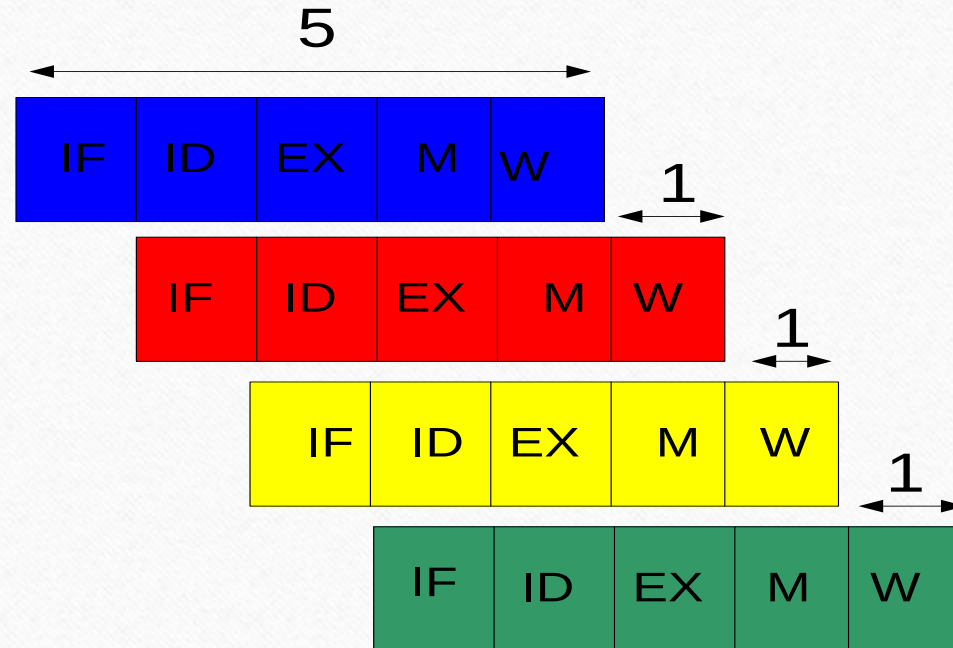
---

- In pipelined architecture,
- Multiple instructions are executed in parallel.
- This style of executing the instructions is highly efficient.

- In pipelined architecture,
- The hardware of the CPU is split up into several functional units.
- Each functional unit performs a dedicated task.
- The number of functional units may vary from processor to processor.
- These functional units are called as stages of the pipeline.
- Control unit manages all the stages using control signals.
- There is a register associated with each stage that holds the data.
- There is a global clock that synchronizes the working of all the stages.
- At the beginning of each clock cycle, each stage takes the input from its register.
- Each stage then processes the data and feed its output to the register of the next stage.

# Five stage pipelining

---



Four Pipelined Instructions

# Five stage

---

- *Instruction Fetch Cycle*
- *Instruction Decode/ Register Fetch Cycle*
- *Execution Cycle*
- *Memory Access Cycle*
- *Write-Back Cycle*

# Instruction Fetch

---

- The instruction Fetch (IF) stage is responsible for obtaining the requested instruction from memory.
- The instruction and the program counter (which is incremented to the next instruction) are stored in the IF/ID pipeline register as temporary storage so that may be used in the next stage at the start of the next clock cycle.

# Instruction Decode

---

- Decode the instruction and at the same time read in the values of the register involved. As the registers are being read, do equality test incase the instruction decodes as a branch or jump.
- The offset field of the instruction is sign-extended incase it is needed. The possible branch effective address is computed by adding the sign-extended offset to the incremented PC. The branch can be completed at this stage if the equality test is true and the instruction decoded as a branch.
- The Instruction Decode (ID) stage is responsible for decoding the instruction and sending out the various control lines to the other parts of the processor.
- The instruction is sent to the control unit where it is decoded and the registers are fetched from the register file.

# Execution

---

- Instruction can be decoded in parallel with reading the registers because the register addresses are at fixed locations.
- The Execution (EX) stage is where any calculations are performed.
- The main component in this stage is the ALU.
- The ALU is made up of arithmetic and logic capabilities.

- 
- If a branch or jump did not occur in the previous cycle, the arithmetic logic unit (ALU) can execute the instruction.
  - At this point the instruction falls into three different types:
    - Memory Reference: ALU adds the base register and the offset to form the effective address.
    - Register-Register: ALU performs the arithmetic, logical, etc... operation as per the opcode.
    - Register-Immediate: ALU performs operation based on the register and the immediate value (sign extended).

# Memory and IO

---

- If a load, the effective address computed from the previous cycle is referenced and the memory is read. The actual data transfer to the register does not occur until the next cycle.
- If a store, the data from the register is written to the effective address in memory.
- The Memory and IO (MEM) stage is responsible for storing and loading values to and from memory.
- It also responsible for input or output from the processor.
- If the current instruction is not of Memory or IO type than the result from the ALU is passed through to the write back stage.

# Write Back

---

- Occurs with Register-Register ALU instructions or load instructions.
- Simple operation whether the operation is a register-register operation or a memory load operation, the resulting data is written to the appropriate register.
- The Write Back (WB) stage is responsible for writing the result of a calculation, memory access or input into the register file.

# Advantages and Disadvantages

---

## Advantages:

- More efficient use of processor
- Quicker time of execution of large number of instructions

## Disadvantages:

- Pipelining involves adding hardware to the chip
- Inability to continuously run the pipeline at full speed because of pipeline hazards which disrupt the smooth execution of the pipeline.

# Hazards

---

- Hazards cause the pipeline to stall because of some conflict in the pipeline.

Prevents the next instruction to be executed.

Types:

1. Structural hazards- contention for same hardware resource
2. Data hazards- dependency between instructions
3. Control hazards- branch/jump instruction stall the pipeline until the target address is loaded in PC.

# Performance of a pipelined execution

The following parameters serve as criterion to estimate the performance of pipelined execution-

- Speed Up
- Efficiency
- Throughput

## Speed Up

It gives an idea of “**how much faster**” the pipelined execution is as compared to non-pipelined execution.

$$\text{Speed Up (S)} = \frac{\text{Non-pipelined execution time}}{\text{Pipelined execution time}}$$

---

## Efficiency

The efficiency of pipelined execution is calculated as-

$$\text{Efficiency } (\eta) = \frac{\text{Speed Up}}{\text{Number of stages in Pipelined Architecture}}$$

OR

$$\text{Efficiency } (\eta) = \frac{\text{Number of boxes utilized in phase-time diagram}}{\text{Total number of boxes in phase-time diagram}}$$

---

## Throughput

Throughput is defined as number of instructions executed per unit time.

$$\text{Throughput} = \frac{\text{Number of instructions executed}}{\text{Total time taken}}$$

- Cycle time = Delay offered by one stage including the delay due to its register
- Cycle time = Maximum delay offered by any stage including the delay due to its register
- Frequency of the clock ( $f$ ) =  $1 / \text{Cycle time}$
- Non-pipelined execution time  

$$= \text{Total number of instructions} \times \text{Time taken to execute one instruction}$$


---


$$= n \times k \text{ clock cycles}$$
- Pipelined execution time  

$$= \text{Time taken to execute first instruction} + \text{Time taken to execute remaining instructions}$$

$$= 1 \times k \text{ clock cycles} + (n-1) \times 1 \text{ clock cycle}$$

$$= (k + n - 1) \text{ clock cycles}$$
- Speed up  

$$= \text{Non-pipelined execution time} / \text{Pipelined execution time}$$

$$= n \times k \text{ clock cycles} / (k + n - 1) \text{ clock cycles}$$

$$= n \times k / (k + n - 1)$$

$$= n \times k / n + (k - 1)$$

$$= k / \{ 1 + (k - 1)/n \}$$